Dossier de rendu : GitOps & CI/CD

Nom : VINCENTE

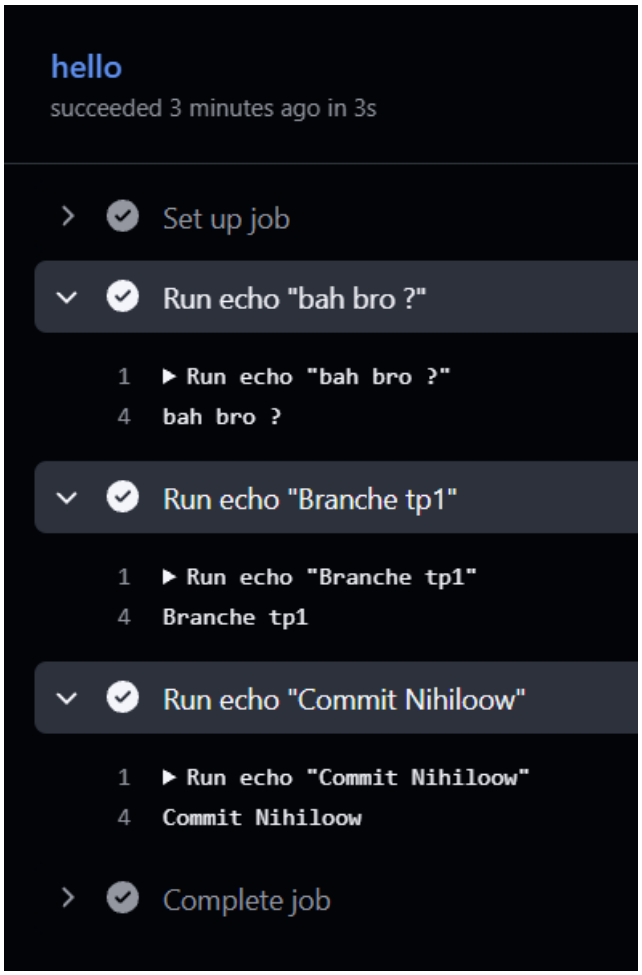
Prénom : Nathan

Date : Début 02/11/26 → Fin 06/11/26

Github : https://github.com/Nihiloow/tasks-api | https://github.com/Nihiloow/tasks-config

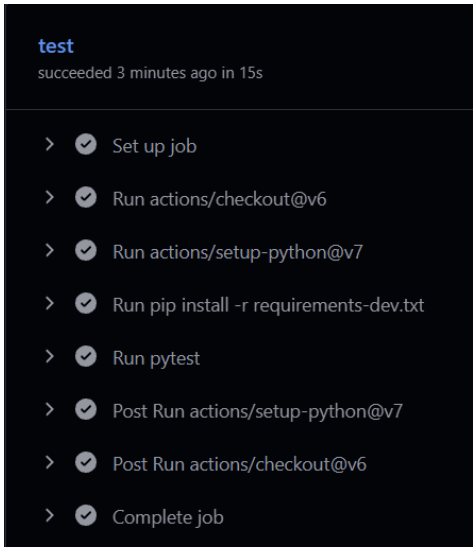
---

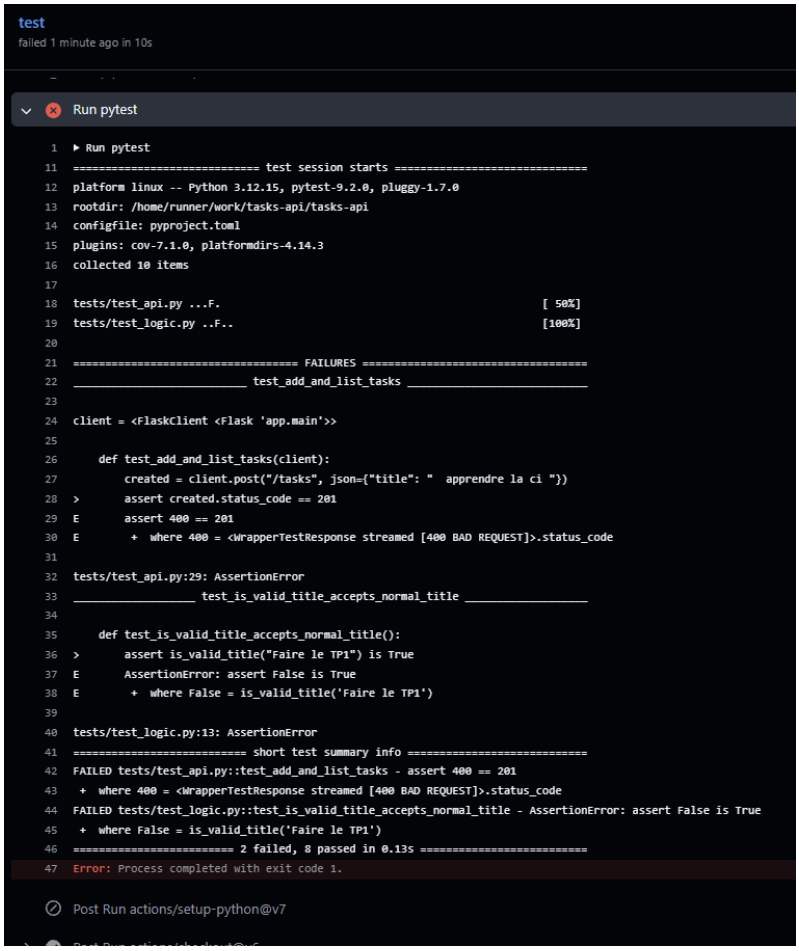
TP n°1

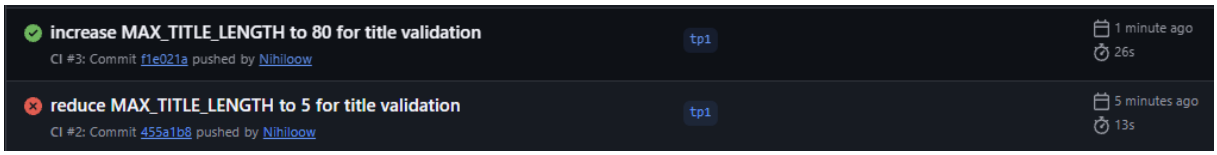


Questions :

- Le run a duré 3s. On peut observer les logs de chaque étape en dépliant les contenus de hello (en dépliant un « Run echo » par exemple)

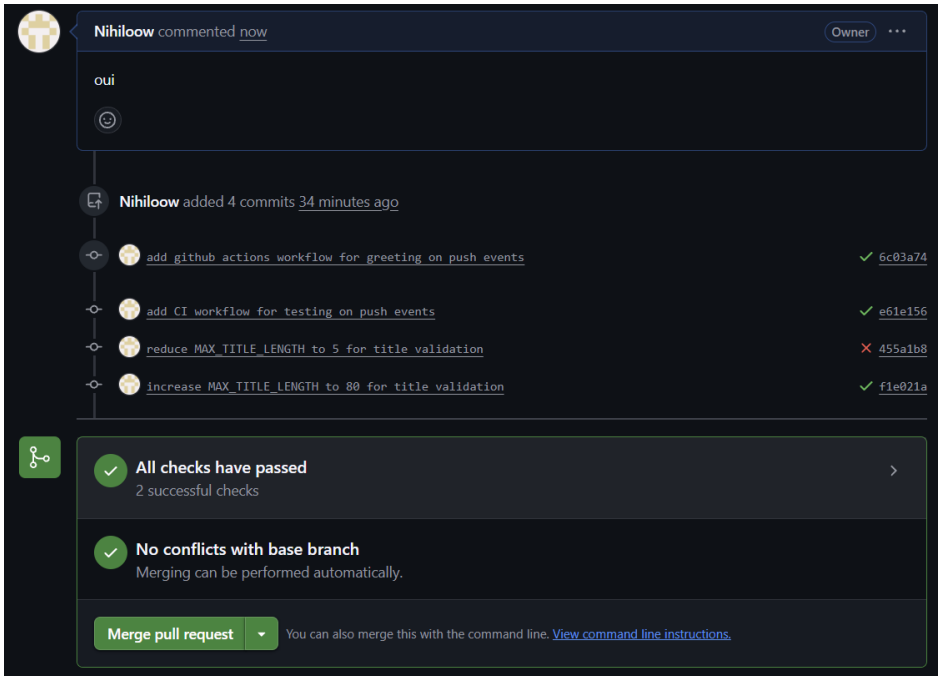


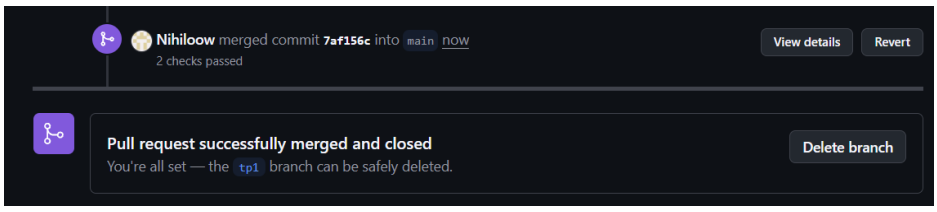


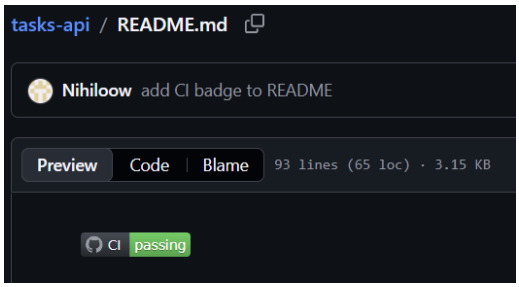


Questions :

- Run pytest a échoué
- Pytest a échoué lamentablement dans le workflow CI dans test_logic.py ligne 13 puis dans test_api.py ligne 29.
- Le log de l'étape "run pytest" montre où l'étape a eu une erreur.



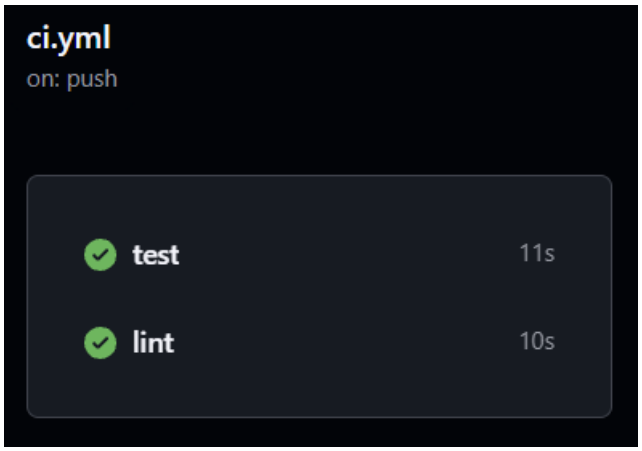


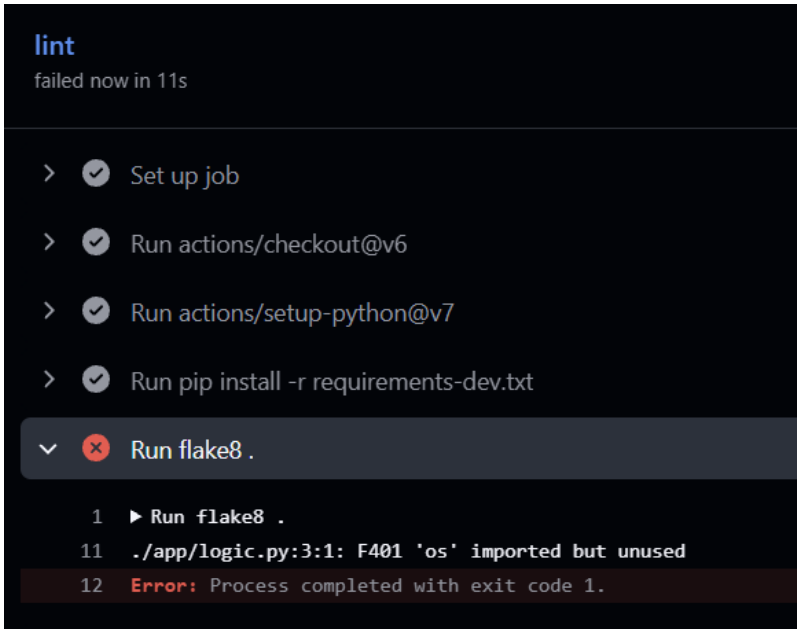


Questions :

Pour que main contienne toujours du code qui fonctionne : avec une CI rouge, on fusionnerait du code cassé que toute l'équipe récupérerait.

TP n°2



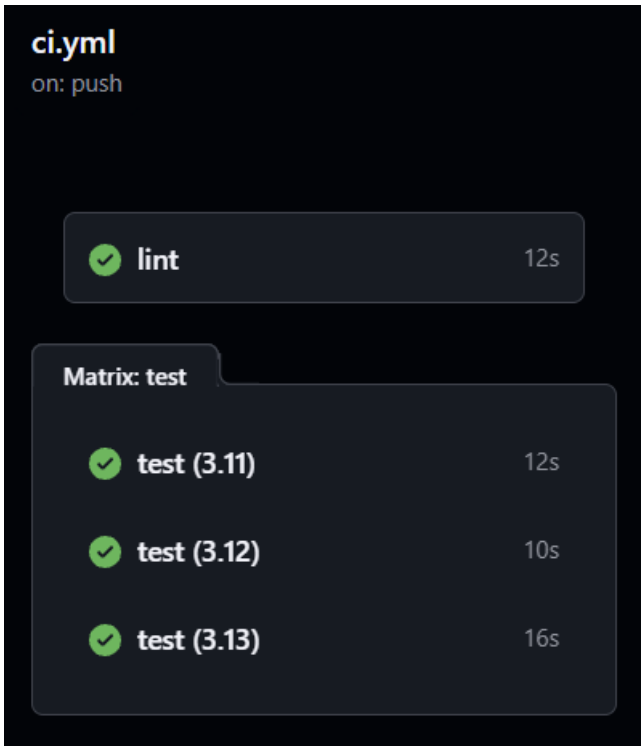


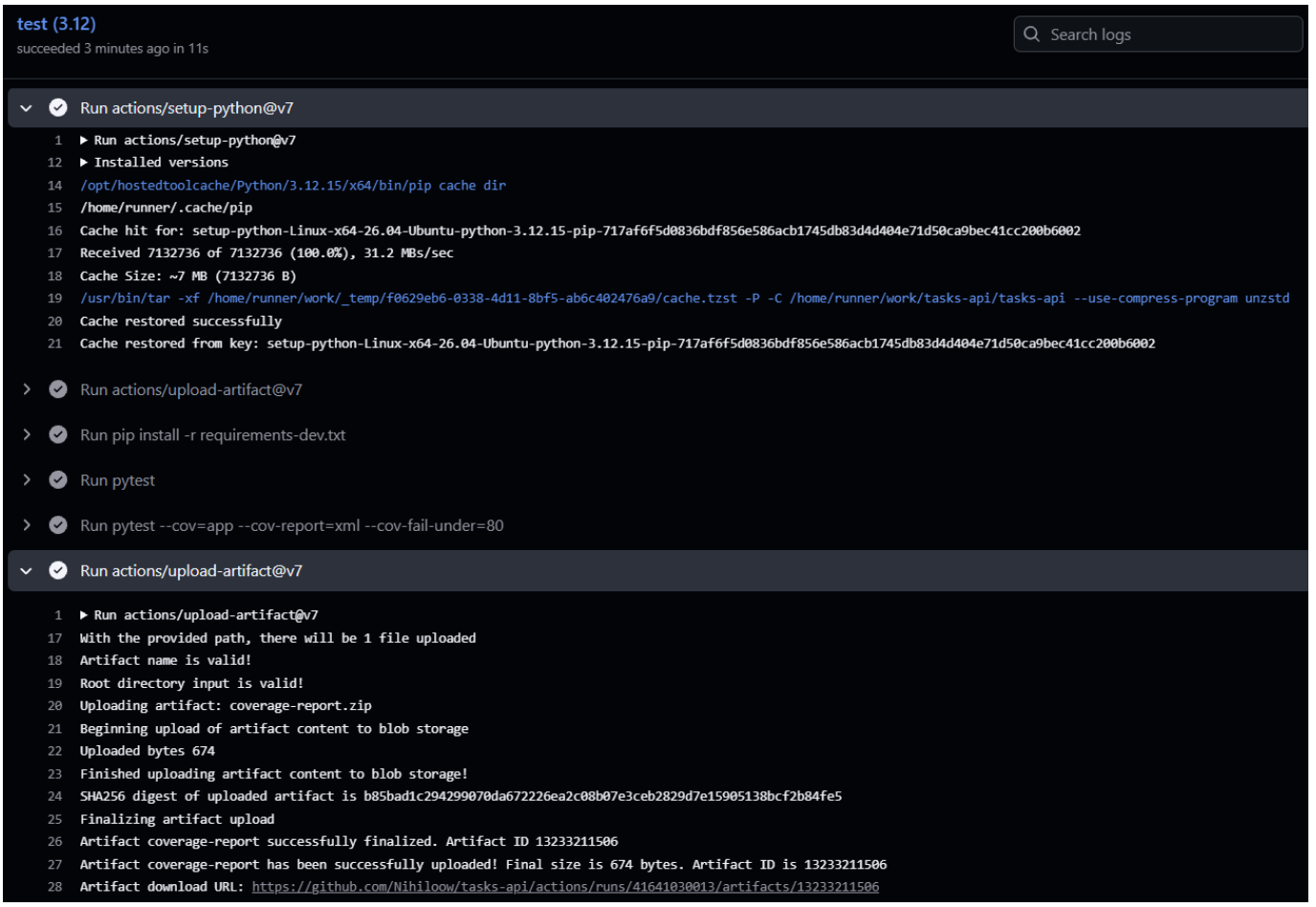
Questions :

Flake8 vérifie dans un fichier que tout le code est utilisé et ne contient pas d'erreur notable.

Black réécrit un fichier de sorte qu'il respecte un stylé défini (il formatte le fichier comme Prettier en gros)

En les séparant, ça permet d’identifier plus facilement si un problème est lié à une erreur de code où ne respecte pas le format imposé par black.



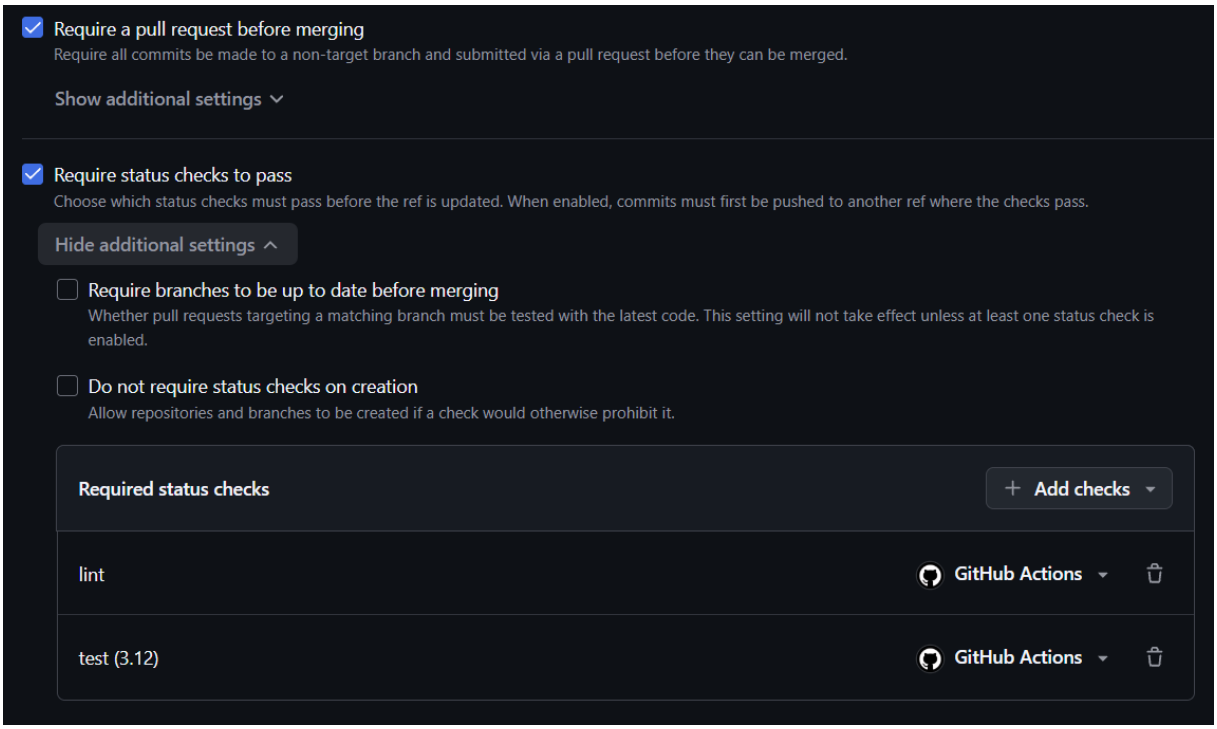


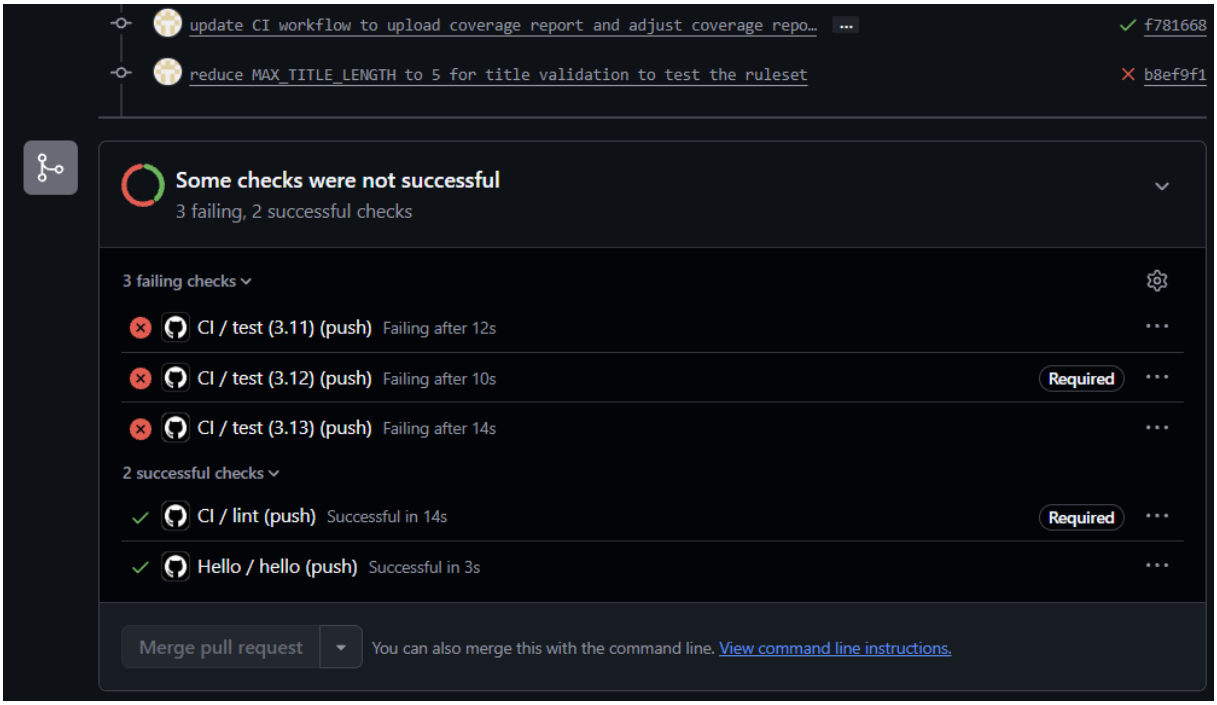
Questions :

- Tester plusieurs versions permet de vérifier et de donner une liste des versions supportées par le projet.

- Le cache nous permet de gagner du temps et de la ressource.

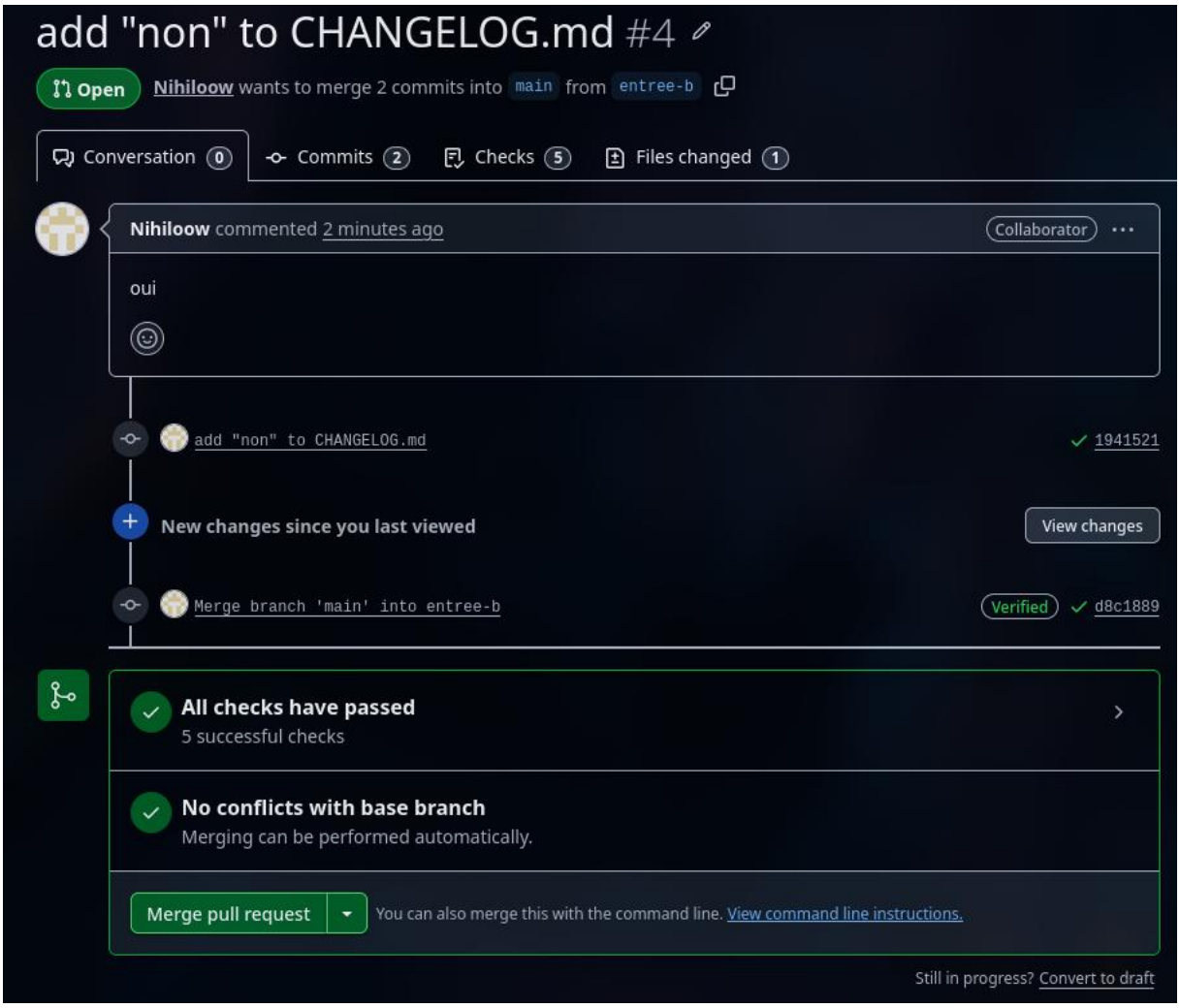
- Un artéfact est un fichier produit par un job et conservé après le run.

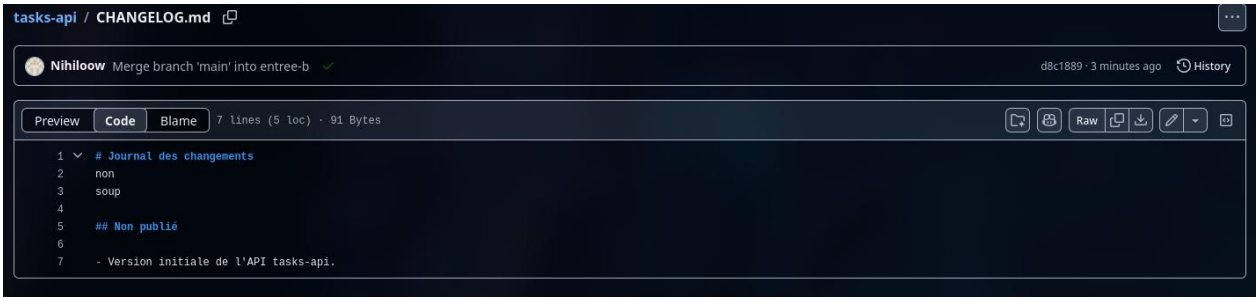




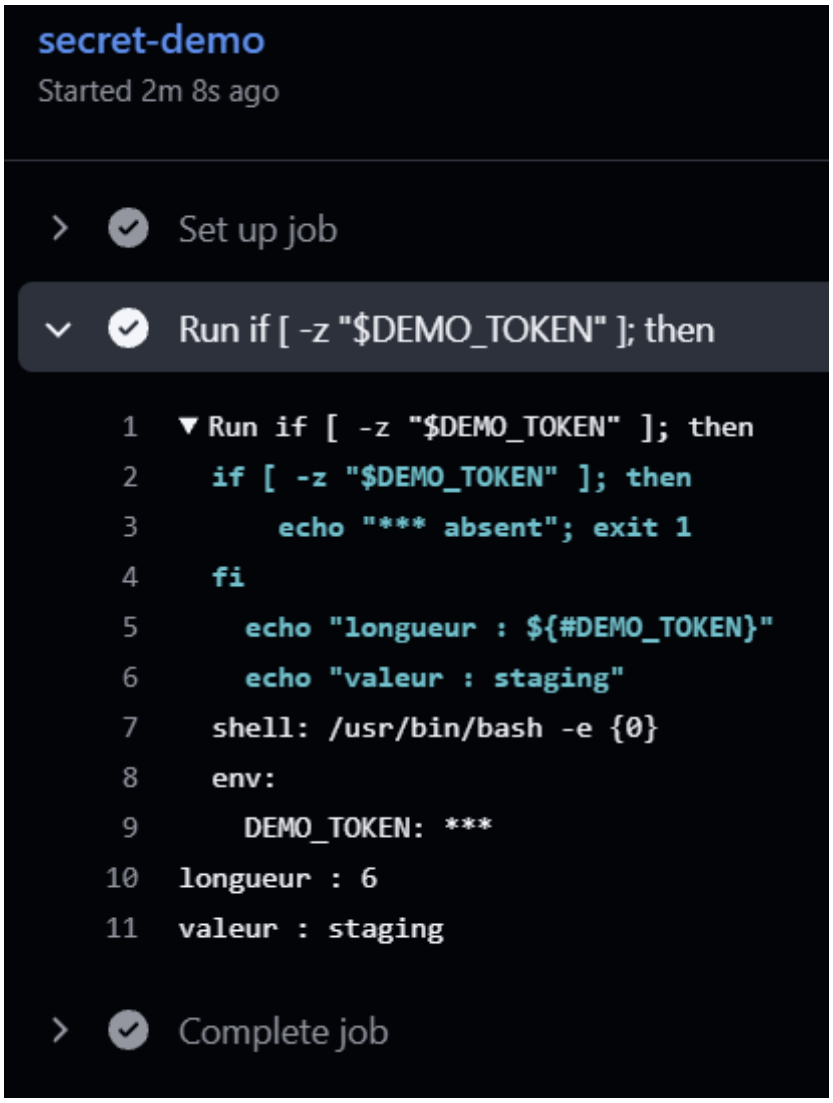
Questions :

- Pour limiter les risques d'envoyer du code qui casse le projet. C'est un premier filtre





Git ne peut pas résoudre le conflit car il ne peut pas prioriser une branche à une autre.
La CI vérfie le code après le merge.



Questions :

- Github masque les variables d'environement de son côté car ce sont des données sensibles.

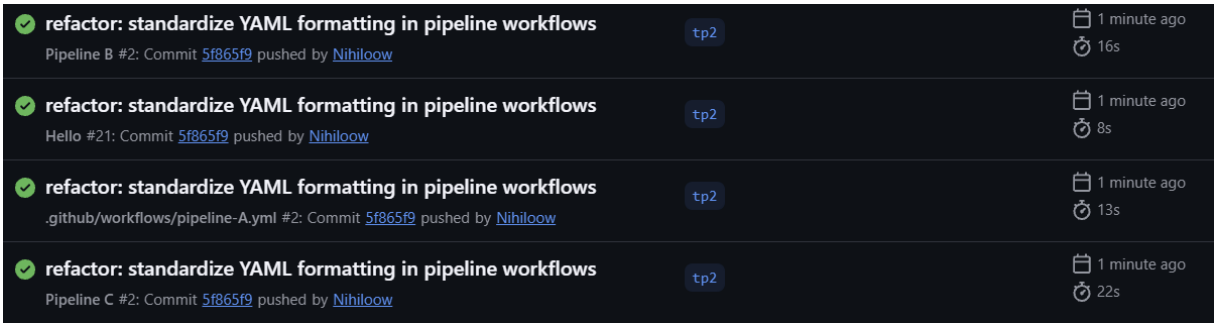
- Si on commite le secret dans le dépôt, le secret est compromis et il faut désactiver toute clé dévoilée.

- Il faut utiliser une variable si nous avons besoin que tout le monde y ait accès, sinon il faut utiliser un secret.

Avant :



Après :



Questions :

A : un espace en trop ligne 12

B : oubli de la ligne “uses: actions/checkout@v6” ligne 6

C : oubli des guillemets dans la matrices des versions de Python ligne 9

- Pour les retrouver, nous avons regardé les logs de chaque job.

- Ces erreurs auraient pu être évité en utilisant les commandes de vérifications (flake8, black)

TP n°3

```
(.venv) PS C:\Users\natha\Documents\Coda\CODA_B2\GitOps\tasks-api> docker build -t tasks-api:local
>> docker run --rm -p 8000:8000 tasks-api:local
>>
[+] Building 29.8s (11/11) FINISHED
linux
```

```
(.venv) PS C:\Users\natha\Documents\Coda\CODA_B2\GitOps\tasks-api> curl.exe http://localhost:8000/health
{"status":"ok"}
(.venv) PS C:\Users\natha\Documents\Coda\CODA_B2\GitOps\tasks-api> curl.exe http://localhost:8000/version
{"version":"dev"}
```

Question :

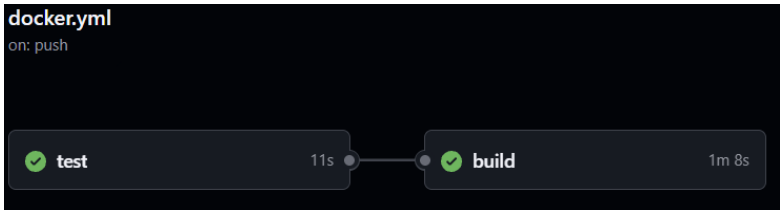
Une image contient les données fixes qui sont utilisé par le conteneur quand on le lance.

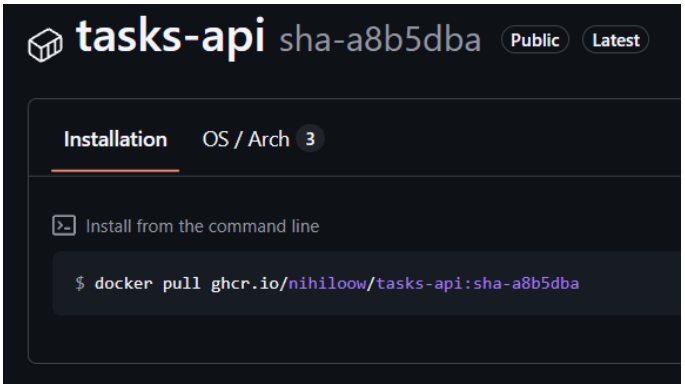
Le fichier .dockerignore permet de ne pas copier les fichiers sensibles dans le conteneur.

```
(.venv) PS C:\Users\natha\Documents\Coda\CODA_B2\GitOps\tasks-api> curl.exe http://localhost:8000/health
{"status":"ok"}
(.venv) PS C:\Users\natha\Documents\Coda\CODA_B2\GitOps\tasks-api> curl.exe http://localhost:8000/version
{"version":"local"}
```

Question :

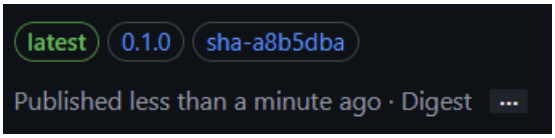
Le fichier docker-compose.yml permet de lancer la même commande avec les mêmes paramètres à chaque fois.





Question :

Il n’y a aucune utilité à créer le package si les tests ne passent pas avant. Il faut éviter le tag “latest” en production car les dernières images ne sont pas obligatoirement optimisées et peuvent contenir des problèmes.

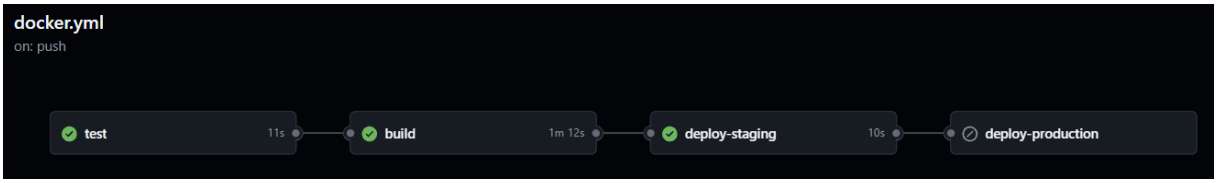


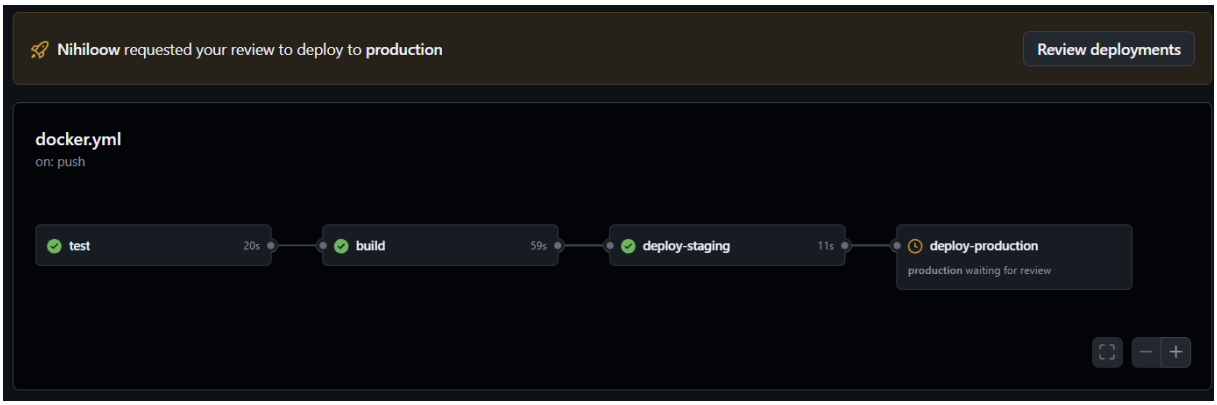

```
(.venv) PS C:\Users\natha\Documents\Coda\CODA_B2\GitOps\tasks-api> docker run --rm -p 8000:8000 ghcr.io/nihiloow/tasks-api:0.1.0
>>
Unable to find image 'ghcr.io/nihiloow/tasks-api:0.1.0' locally
0.1.0: Pulling from nihiloow/tasks-api
9a62a3c3f631: Pull complete
49effa291546: Pull complete
aae8c60d4b9e: Pull complete
b34cd5e23ffa: Pull complete
3e5de57508e2: Pull complete
Digest: sha256:709ce72ffcefe551a6e666c232d313db10bbe25fc8ee2ea7c0039026e538ab1a
Status: Downloaded newer image for ghcr.io/nihiloow/tasks-api:0.1.0
[2026-11-04 14:58:08 +0000] [1] [INFO] Starting gunicorn 26.2.0
[2026-11-04 14:58:08 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
[2026-11-04 14:58:08 +0000] [1] [INFO] Using worker: sync
[2026-11-04 14:58:08 +0000] [7] [INFO] Booting worker with pid: 7
[2026-11-04 14:58:09 +0000] [1] [INFO] Control socket listening at /home/appuser/.gunicorn/gunicorn.ctl
```

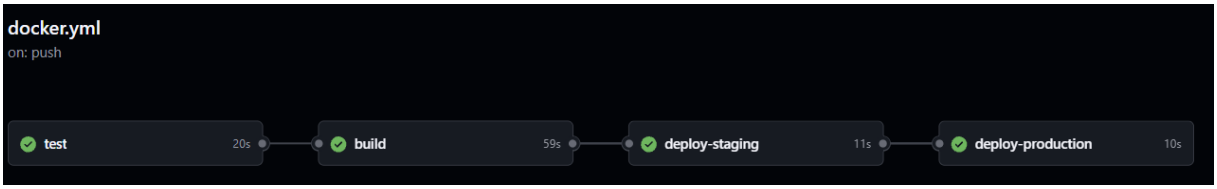
```
(.venv) PS C:\Users\natha\Documents\Coda\CODA_B2\GitOps\tasks-api> curl.exe http://localhost:8000/version
{"version":"0.1.0"}
```

Question :

Il n’y a aucune utilité à créer le package si les tests ne passent pas avant. Il faut éviter le tag “latest” en production car les dernières images ne sont pas obligatoirement optimisées et peuvent contenir des problèmes.

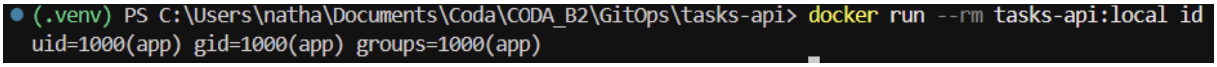






Question :

Le test de fumée permet de vérifier que l'application démarre et répond. L'approbation humaine permet de vérifier que la nouvelle version déployée ne contient pas de code malicieux.

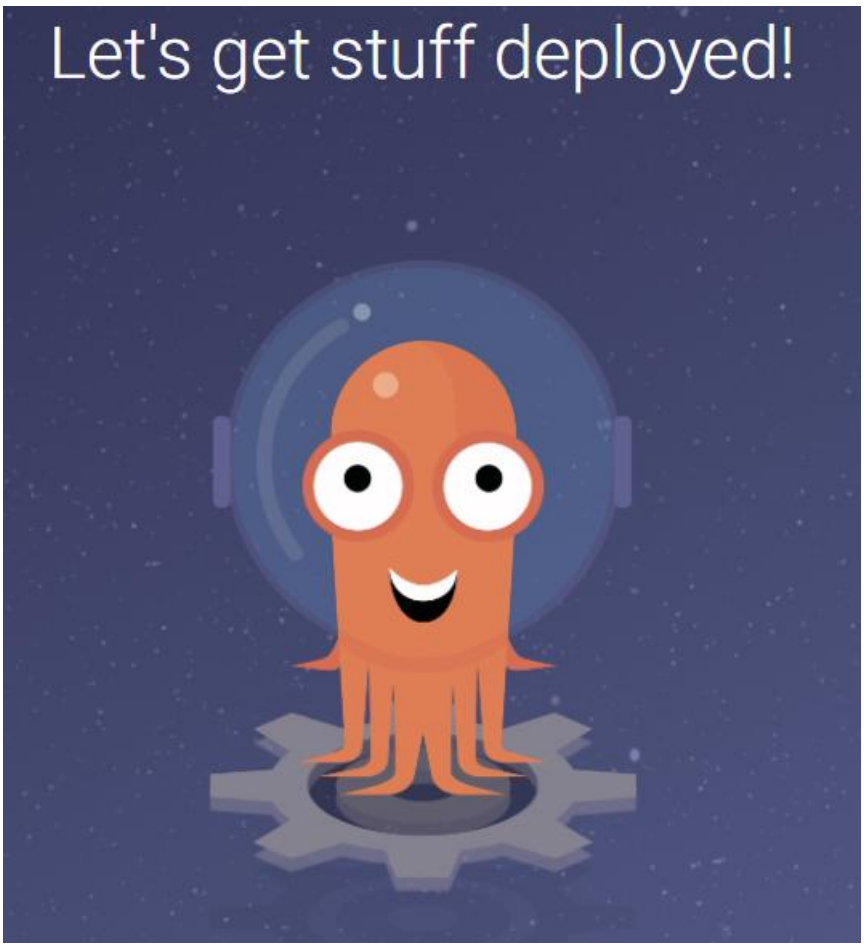


Question :

Un conteneur n'est pas une machine virtuelle. Un code malicieux peut accéder au reste des fichiers en dehors du conteneur car le noyau est le même. Donner les permissions root au conteneur est une faille de sécurité pour la machine.

TP n°4

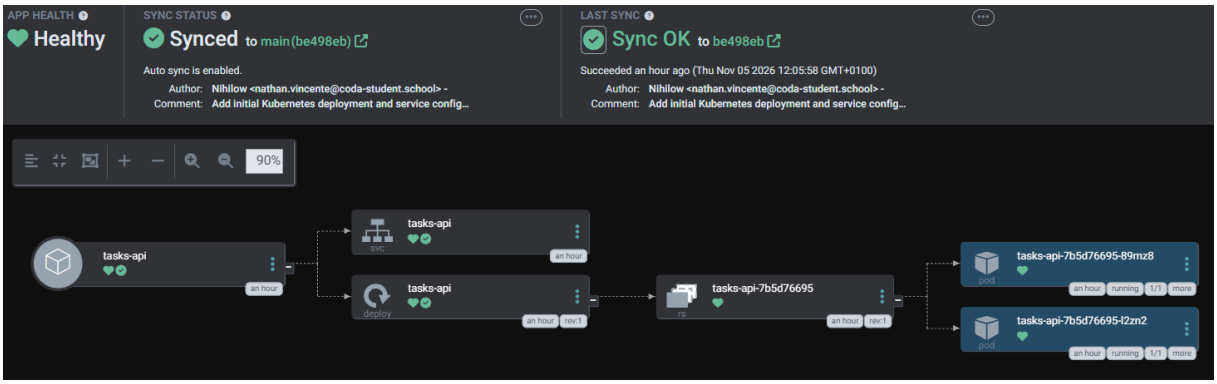
```
(.venv) PS C:\Users\natha\Documents\Coda\CODA_B2\GitOps\tasks-api> kubectl get nodes
NAME                 STATUS   ROLES           AGE   VERSION
gitops-control-plane   Ready    control-plane   12m   v1.37.0
(.venv) PS C:\Users\natha\Documents\Coda\CODA_B2\GitOps\tasks-api> kubectl get pods -n argocd
NAME                                                READY   STATUS    RESTARTS      AGE
argocd-application-controller-0                     1/1     Running   0             12m
argocd-applicationset-controller-794bb95c9d-wdghr   1/1     Running   0             12m
argocd-dex-server-66cf64bfcf-5lz6q                  1/1     Running   1 (10m ago)   12m
argocd-notifications-controller-5878c8d644-msp9m    1/1     Running   0             12m
argocd-redis-5869b7547-mg6h2                        1/1     Running   0             12m
argocd-repo-server-7d55cc466b-5r9z4                 1/1     Running   0             12m
argocd-server-85b849c688-rxh42                      1/1     Running   0             12m
```



Question :

Un pod est un ou un groupe de conteneurs qui tournent ensemble. Ils sont ephémères.

Les namespaces permettent de faciliter l'organisation des ressources.



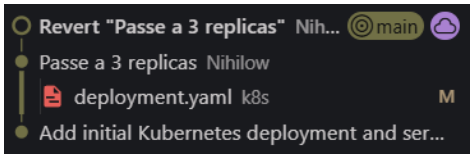

```
(.venv) PS C:\Users\natha\Documents\Coda\CODA_B2\GitOps\tasks-api> curl.exe http://localhost:8081/version
{"version":"0.1.0"}
```

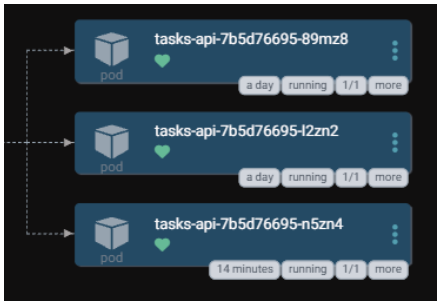
Questions :

Le dépôt de l'application déclenche la CI à chaque push. Le dépôt de configuration, surveillé par Argo CD, permet de déployer sans relancer la CI à chaque changement de configuration. Les développeurs modifient le code, mais seuls le pipeline et les personnes autorisées modifient ce qui part en production, et l'historique de la config sert de journal des déploiements.

Questions :

Avant la modification, 3 pods tournaient. Après le kubectl scale à 5, 2 nouveaux pods sont apparus : il y a donc eu 5 pods pendant environ 1 seconde, mais les 2 nouveaux n'ont jamais été prêts (0/1). Argo CD a presque tout de suite détecté l'écart avec Git et, grâce à selfHeal: true, a remis 3 replicas : les 2 pods en trop sont passés en Terminating puis ont été supprimés. Les 3 pods d'origine n'ont pas bougé

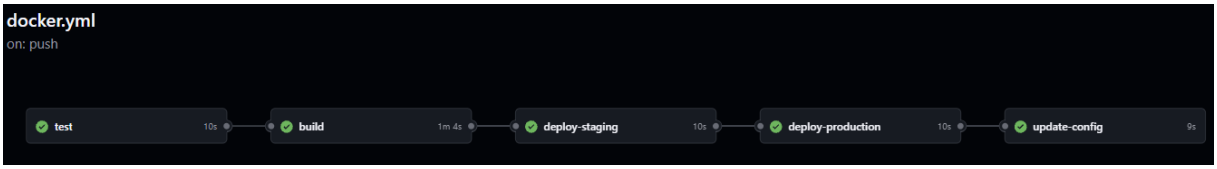


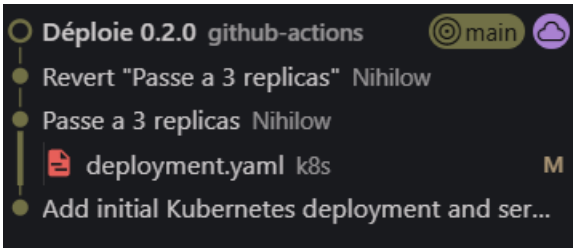


Questions :

selfHeal : si quelqu'un modifie le cluster à la main, Argo CD le remet automatiquement comme dans Git.

Retour arrière : il suffit d'un git revert du dernier commit, et Argo CD remet l'ancienne version.







Questions :

1. Je crée et pousse un tag v0.2.0 → le pipeline de tasks-api se déclenche et lance les tests.
2. Si les tests passent, l'image 0.2.0 est construite et publiée sur GHCR.
3. Elle est déployée en staging, puis attend l'approbation pour la production.
4. Le pipeline met à jour le tag de l'image dans tasks-config par un commit and push.
5. Argo CD voit le changement dans Git et synchronise le cluster → les nouveaux pods démarrent.

Le pipeline pousse l'image vers GHCR et le commit vers tasks-config. Argo CD tire la configuration depuis Git, et le cluster tire l'image depuis GHCR.

En CI/CD, le pipeline se connecte au cluster et y applique lui-même le changement → il détient donc les clés du cluster. En GitOps, le pipeline ne touche jamais au cluster. Les clés restent dans le cluster, et Argo CD corrige en continu tout écart avec Git.

Bilan (je ne savais pas s'il fallait le faire) :

Grâce au tp, j'ai compris comment un simple git push ou git tag peut tester, construire et déployer une application sans aucune action manuelle. Ce que je retiens le plus, c'est la logique GitOps. On ne modifie plus le cluster, on modifie Git, et Argo CD remet le cluster en accord avec le dépôt. Ce qui m'a posé le plus de problèmes, c'est l'indentation YAML et le nom de l'image en minuscules pour GHCR. À chaque fois, lire le log de l'étape en erreur m'a permis de trouver la cause des nombreux problèmes qu'il y a eu. J'ai quand même eu besoin de l'IA pour m'expliquer certaines erreurs git.